

ARCHITECTURE STUDY

OpenAI Realtime Audio Support for Geppetto

Browser audio transport, backend control, session design, tools, persistence, and rollout

Research date: 2026-08-11

Codebase reviewed: `geppetto-sessionstream.zip`

Archive SHA-256: `26628f8b3458050d493adebe157f7fdFDA10c4ad8d15545412c5593314faafd1`

Primary recommendation

Use direct browser-to-OpenAI WebRTC for the audio media path, an authenticated Geppetto endpoint for session bootstrap, and an OpenAI sideband WebSocket from Geppetto for tools, policy, observability, and durable state. Add a provider-neutral long-lived `live` subsystem instead of treating Realtime as another finite `engine.Engine`.

Current as of 2026-08-11. OpenAI model names and protocol details must remain configuration-driven because this API surface continues to evolve.

Executive summary

Geppetto can support OpenAI Realtime audio cleanly, but the implementation should not be forced into its existing finite inference abstraction. The current `engine.Engine` contract accepts one `Turn` and returns one updated `Turn`; `session.ExecutionHandle` represents one cancelable, waitable inference; and `toolloop.Loop` repeatedly invokes that finite engine. A Realtime call is structurally different: it is a long-lived, bidirectional session containing many user speech turns, model responses, interruptions, function calls, configuration updates, and transport events.

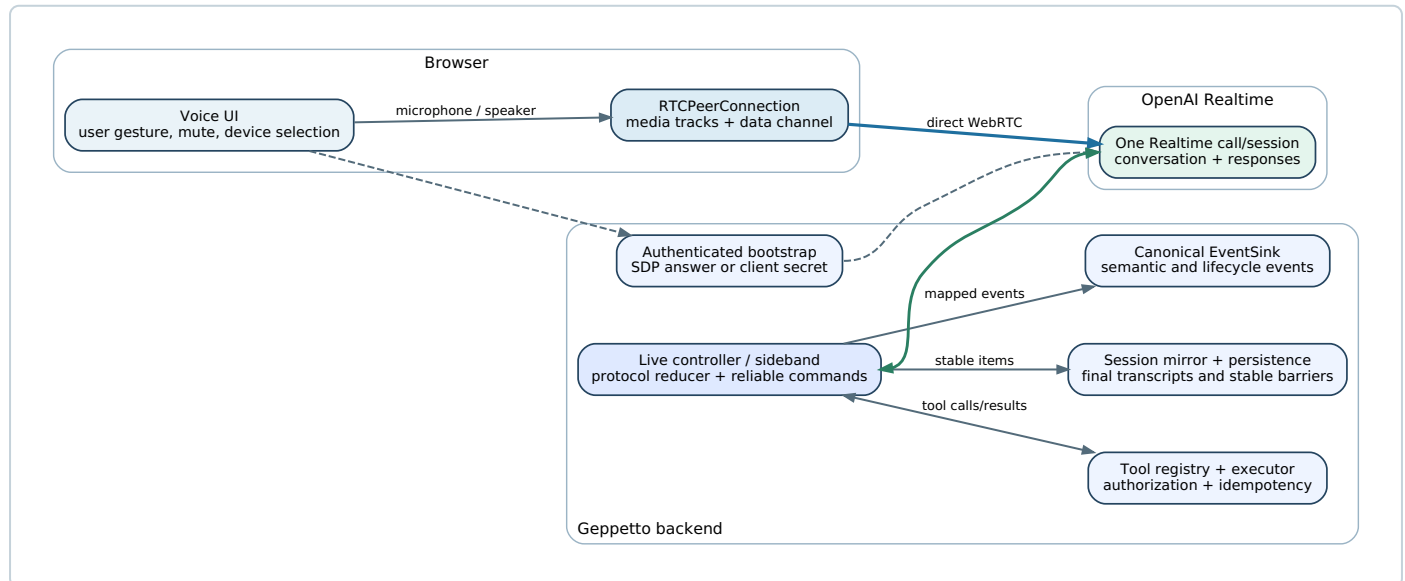
The correct design is a parallel provider-neutral `pkg/inference/live` subsystem. It should reuse Geppetto's strongest existing concepts - stable `SessionID` values, canonical event correlation, `EventSink`, tool registry/executor, Turn persistence, settings, and JavaScript-facing session vocabulary - while introducing a separate lifecycle and reliable command path for duplex sessions.

Direct answers

- 1. Can the browser stream microphone audio directly to OpenAI?** Yes. For browser and mobile clients, OpenAI recommends WebRTC. The browser supplies a microphone media track to `RTCPeerConnection`, receives model audio as a remote media track, and uses a data channel for JSON control events. It does not manually Base64-encode microphone chunks in this topology. [OAI-WEBRTC](#) [OAI-CONVERSATIONS](#)
- 2. Does all audio need to pass through the application's backend?** No. In the recommended topology, audio flows between the browser and OpenAI. Geppetto remains involved for authentication/bootstrap and attaches to the same Realtime call through a server-side sideband WebSocket for tools and control. [OAI-SIDEBAND](#)
- 3. Is any backend required?** For a secure production browser application, yes, but it can be very small. A trusted endpoint either mints a short-lived Realtime client secret or forwards the browser's SDP offer through the unified `/v1/realtime/calls` interface. The permanent OpenAI API key must remain on the server. [OAI-WEBRTC](#) [OAI-CLIENT-SECRETS](#)
- 4. Can the browser connect over WebSocket instead?** Technically yes with an ephemeral token, but OpenAI recommends WebRTC for browser/mobile clients. Browser WebSocket audio requires manual capture, resampling, PCM framing, Base64 encoding, playback buffering, and interruption truncation. [OAI-WEB SOCKET](#) [OAI-CONVERSATIONS](#)
- 5. Should Geppetto support a backend media proxy?** Yes as a second transport, not the default. It is justified for telephony, recording/compliance, transcoding, media inspection, centralized routing, or non-WebRTC environments. It adds latency, bandwidth, CPU, failure modes, and a difficult playback/truncation implementation.
- 6. Can the proposed feature benefit from both existing Geppetto designs?** Yes. The session-centered API should own identity, lifecycle, snapshots, and persistence. The streaming `EventEmitter/EventSink` design should expose semantic deltas and lifecycle events. Raw audio must remain on a dedicated media path rather than being emitted through the general event bus.

7. **Should Geppetto support both voice architectures?** Yes. Expose both a native speech-to-speech Realtime mode and a chained mode that uses live transcription -> existing Geppetto text agent/tool loop -> streaming speech generation. OpenAI documents both as valid architectures with different control/latency tradeoffs. [OAI-VOICE-AGENTS](#)

Recommended production shape



The media plane and control plane are intentionally separate:

- **Media plane:** browser microphone and speaker audio over WebRTC directly to the OpenAI Realtime call.
- **Bootstrap plane:** a short authenticated HTTP exchange with Geppetto, using either the unified SDP interface or a short-lived client secret.
- **Control plane:** a Geppetto sideband WebSocket attached by OpenAI `call_id`, used for session updates, tool execution, canonical event mapping, metrics, and persistence.
- **Durable plane:** only stable semantic state is written to Geppetto Turns. Raw audio deltas and partial provider events are not treated as durable conversation state.

Main design decisions

Decision	Recommendation	Reason
Public abstraction	Add <code>live.Session</code> / <code>live.Controller</code>	A Realtime call is not one finite inference.
Browser transport	Direct WebRTC	Lowest implementation burden and best browser media behavior.
Backend role	Bootstrap + sideband	Keeps keys/tools private without proxying audio.
Strict client isolation	Backend-owned WebSocket proxy when required	Direct browser sessions give the client protocol access; instructions are not an authorization boundary.
Tool execution	Existing registry/executor through a new reliable <code>ToolDispatcher</code>	Reuse schemas and execution, but not the finite <code>toolloop.Loop</code> orchestrator.
Events	Reuse canonical events; add live/audio lifecycle events	Preserves observability and JS integration.
Raw audio	Never through default <code>EventSink</code>	Prevents high-volume copies, memory pressure, and accidental logging.
Persistence	Final transcripts, completed items, tool barriers, close snapshots	Partial deltas are reorderable and interruption-sensitive.
Voice product modes	<code>native_realtime</code> and <code>chained</code>	Natural low latency versus explicit text control and existing-agent reuse.
Model selection	Configuration profile, not a hard-coded string	Current docs use both model families and point releases; the surface will change.

Contents

- 1. Scope, method, and limitations
- 2. What the existing Geppetto architecture provides
- 3. How the OpenAI Realtime API behaves
- 4. Browser-to-API transport study
- 5. Recommended target architecture
- 6. Supporting both native and chained voice
- 7. Detailed Go package and interface design
- 8. Browser client design
- 9. Backend endpoint and sideband design
- 10. Events, correlation, tools, and persistence
- 11. Security and privacy model
- 12. Reliability, interruption, and long-session behavior
- 13. Observability, capacity, and cost
- 14. Testing strategy
- 15. Incremental rollout plan
- 16. Decision record and implementation checklist
- 17. Appendices and references

1. Scope, method, and limitations

1.1 Questions studied

This report addresses four linked questions:

- How should OpenAI Realtime audio fit into the uploaded Geppetto codebase?
- Which existing session and streaming-event concepts can be reused, and where are new abstractions required?
- How can a browser stream microphone audio to the API, and when must audio pass through an application backend?
- How should the design handle tools, security, persistence, interruption, observability, testing, and migration?

1.2 Review method

The analysis combined:

1. Static inspection of the uploaded repository, including engine, session, event, Turn, tool-loop, OpenAI adapter, settings, transcription, and JavaScript module code.
2. Review of the repository's design notes for the session-centered JavaScript API and streaming EventEmitter work.
3. Review of current official OpenAI documentation for WebRTC, WebSocket, client secrets, sideband control, conversation events, VAD, transcription, voice architectures, models, and costs.
4. Architectural comparison of direct WebRTC, unified SDP bootstrap, direct browser WebSocket, backend WebSocket media proxy, and chained voice.

The archive contains 2,286 files, including 503 Go files and 1,444 Markdown files. It does not contain `.git` metadata, so no commit hash could be identified.

1.3 Validation limitation

A full `go test ./...` run could not be completed in the isolated environment because `go.mod` requests Go toolchain `go1.26.5`, which the local Go command attempted to download, and network access was unavailable. Static review also found imports/references to `pkg/inference/toolloop/enginebuilder`, while that package directory is absent from the supplied archive. These facts are baseline caveats, not evidence that the proposed design or the rest of the codebase fails to compile in its normal development environment.

1.4 Source notation

- Repository citations use `path:line-line`, based on the supplied archive.
- OpenAI citations such as [OAI-WEBRTC](#) refer to official documentation listed in the References section.
- Statements labeled **recommendation** or **inference** are architectural judgments derived from the reviewed sources rather than claims made by those sources.

2. What the existing Geppetto architecture provides

2.1 The current engine is deliberately finite

`pkg/inference/engine/engine.go:9-15` defines:

```
type Engine interface {
    RunInference(ctx context.Context, t *turns.Turn) (*turns.Turn, error)
}
```

The comments are explicit: an engine processes a Turn, returns an updated Turn, and does not own tool orchestration. The Chat Completions and Responses adapters may consume provider streaming internally, but their public behavior is still finite: start one provider call, reduce deltas, and return a final Turn.

This is a good abstraction for text and batch-like inference. It is a poor lifecycle match for Realtime because a Realtime connection:

- remains open across many speech turns;
- accepts commands at arbitrary times;
- emits server events independently of a single method invocation;
- can have overlapping response, transcript, and tool streams;
- supports interruptions and mutable session configuration;
- terminates due to explicit close, connection failure, policy, or maximum session age rather than completion of one response.

Conclusion: do not add an `OpenAIRuntimeEngine` that blocks inside `RunInference` for the lifetime of a call, and do not redefine `RunInference` to sometimes mean one response and sometimes mean one hour-long duplex session.

2.2 The current Session is valuable, but its active unit is one inference

`pkg/inference/session/session.go:21-35` gives Geppetto a stable `SessionID`, append-only Turn snapshots, and an invariant of one active inference. `AppendNewTurnFromUserPrompts` clones the latest Turn and assigns a new Turn ID (`session.go:53-103`). `StartInference` builds one blocking runner, launches it in a goroutine, and returns an `ExecutionHandle` (`session.go:189-281`).

`pkg/inference/session/execution.go:13-84` confirms that the handle represents one in-flight inference with `Cancel`, `Wait`, and `IsRunning`.

These semantics should remain intact for text agents and chained voice. Native Realtime needs a sibling abstraction with a different active-unit invariant:

- one active provider connection per live session;
- zero or one active default-conversation response at a time in the common case;

- potentially multiple out-of-band responses;
- independent transcript and tool operations;
- explicit connection state and terminal error.

The existing `SessionID` should still be the canonical Geppetto identifier. A live session can produce stable Turn snapshots at semantic barriers, but should not append a new Turn for every protocol delta.

2.3 The event system is a strong reuse point

`pkg/events/sink.go:3-18` describes `EventSink` as the integration point for streaming deltas, WebSocket broadcasts, logs, traces, and metrics. It also warns against constructing durable state from partial events. `pkg/events/context.go:15-48` fans events out on a best-effort basis and ignores sink errors. `pkg/events/correlation.go:3-20` provides canonical IDs:

- `SessionID`
- `RunID`
- `TurnID`
- `ProviderCallID`
- `SegmentID`
- `ToolCallID`

The correlation comments correctly state that provider-native response/item IDs belong in adapter state or debug payloads, not as canonical routing keys.

This is almost exactly what Realtime needs for semantic observability. It is not sufficient for control. Since sinks are best-effort, a tool result or `conversation.item.truncate` command must never depend on an event consumer successfully receiving an `EventSink` event. The live subsystem therefore needs two paths:

- **reliable internal command/effect path** for protocol correctness;
- **best-effort canonical event path** for UI, telemetry, logs, and subscribers.

2.4 The Turn model is text/tool oriented

`pkg/turns/block_kind_gen.go:13-21` defines user text, LLM text, tool call/use, system, reasoning, and other blocks. `pkg/turns/helpers_blocks.go:14-40` supports text and images, but not audio. `Turn.Clone` copies block metadata and payload maps only shallowly for nested values (`pkg/turns/types.go:28-62`).

The initial implementation should not introduce raw audio blocks. Raw PCM/Base64 would make Turns huge, leak sensitive media into ordinary persistence/logging, and interact badly with shallow nested values. If recording is later required, store an immutable external media reference:

```
type MediaRef struct {
    URI          string
    MIMETYPE     string
    SHA256       string
    DurationMS   int64
}
```

```

SampleRate    int
Channels      int
ProviderItemID string // provenance/debug only
}

```

A transcript block may carry a `MediaRef` as metadata, but the Turn itself should store semantic content, not the byte stream.

2.5 Tool infrastructure is reusable; the current loop is not

`pkg/inference/toolloop/loop.go:92-174` repeatedly invokes a finite engine, extracts tool calls, executes them, appends tool results, and invokes the model again. That orchestration cannot own a Realtime call because the provider session itself already owns the conversation and response lifecycle.

The following parts remain valuable:

- thread-safe tool registry (`pkg/inference/tools/registry.go`);
- tool executor interface (`pkg/inference/tools/executor.go`);
- JSON schema generation and definitions;
- canonical tool events;
- authorization/deadline middleware that exists around tool execution.

A new `live.ToolDispatcher` should consume provider function-call events, execute through the existing registry/executor, and send the result back over the live transport.

2.6 Settings and factory boundaries should stay explicit

`pkg/inference/engine/factory/factory.go:105-197` returns finite engines for Chat, Responses, Claude, and Gemini. `pkg/steps/ai/settings/settings-chat.go:22-61` has a `Stream` flag, but this means streaming one HTTP response, not opening a duplex media session.

Adding a branch to `EngineFactory` based on `ApiTypeOpenAIRealtime` would create a return-type mismatch. The better separation is:

```

EngineFactory    -> finite engine.Engine
LiveSessionFactory -> long-lived live.Controller / live.Session

```

A Realtime API type may still be useful for profile selection and CLI/config parsing, but it should not imply that Realtime implements `engine.Engine`.

2.7 Existing transcription code is not live microphone streaming

`pkg/steps/ai/openai/transcribe.go:300-459` uses the legacy file-transcription client. Its so-called streaming mode reads chunks from a file and submits separate transcription requests. That is not equivalent to a persistent Realtime transcription session, does not provide browser microphone transport, and may split encoded container data at unsafe byte boundaries.

For chained voice, add a real Realtime transcription transport based on the current transcription session API. OpenAI's current guide uses a transcription session, streams `input_audio_buffer.append`, and emits incremental and completed transcript events. [OAI-TRANSCRIPTION](#)

3. How the OpenAI Realtime API behaves

3.1 Session, Conversation, Items, and Responses

The provider model is not “one request.” A Realtime Session contains:

- a mutable Session configuration;
- a Conversation containing user/model Items;
- Responses that generate audio or text Items;
- an input audio buffer when using WebSocket or explicit buffer control;
- function-call Items and function-call-output Items;
- lifecycle, transcript, rate-limit, and error events.

The server sends `session.created`, accepts `session.update`, and sends `session.updated`. Most settings can change while connected, but the output voice cannot change after the model has emitted audio. Current documentation caps a Realtime session at 60 minutes. [OAI-CONVERSATIONS](#)

This provider lifecycle implies a long-lived reducer, not a method that reduces one SSE response.

3.2 WebRTC separates media tracks from JSON events

In a browser WebRTC topology:

- the microphone is obtained with `navigator.mediaDevices.getUserMedia`;
- its audio track is added to `RTCPeerConnection`;
- model speech arrives as a remote audio track;
- a WebRTC data channel carries JSON client/server events;
- WebRTC and the service handle media packetization, timing, congestion behavior, and output buffering.

OpenAI recommends WebRTC rather than WebSocket for browser/mobile clients. [OAI-WEBRTC](#)

3.3 WebSocket exposes the low-level audio protocol

Over WebSocket, the application must:

- convert input to a configured audio format;
- Base64-encode chunks into `input_audio_buffer.append` JSON events;
- commit the buffer manually when VAD is disabled;
- receive Base64 audio bytes from `response.output_audio.delta`;
- buffer and schedule playback;
- stop playback on interruption;

- track how much audio actually played;
- send `conversation.item.truncate` with `audio_end_ms` so unplayed speech is removed from conversation state.

OpenAI explicitly notes that `response.output_audio.done` and `response.done` do not contain the audio bytes. [OAI-CONVERSATIONS](#)

3.4 VAD and response control

VAD is enabled by default for supported speech-to-speech sessions. Current modes are:

- `server_vad`: detects speech boundaries from audio activity;
- `semantic_vad`: estimates whether the user has semantically completed the utterance, often improving natural turn-taking at some latency cost;
- `null`: disables turn detection, requiring manual commit and response creation.

The API emits `input_audio_buffer.speech_started` and `input_audio_buffer.speech_stopped`. It can also retain VAD events while disabling automatic response creation, which is useful when Geppetto must run moderation, retrieval, or policy checks before allowing a response. [OAI-VAD](#) [OAI-CLIENT-SECRETS](#)

3.5 Transcripts have different authority levels

There are two important transcript classes:

- **output audio transcript:** transcript associated with model speech;
- **input audio transcription:** a separate asynchronous transcription of user audio.

The input transcription is not the native model's internal audio understanding and should be treated as guidance rather than a byte-for-byte or semantic guarantee of what the model heard. [OAI-CLIENT-SECRETS](#)

This matters for persistence and audits. Geppetto should store transcript source metadata and avoid claiming that a user transcript is the exact model input interpretation.

3.6 Function calls form an event-driven loop

When the model emits a function call, the application executes it and sends:

1. `conversation.item.create` with `item.type = "function_call_output"`, the same `call_id`, and a JSON string output;
2. `response.create` to ask the model to continue.

This is analogous to the existing tool loop, but the orchestration lives inside the ongoing Realtime session. [OAI-CONVERSATIONS](#)

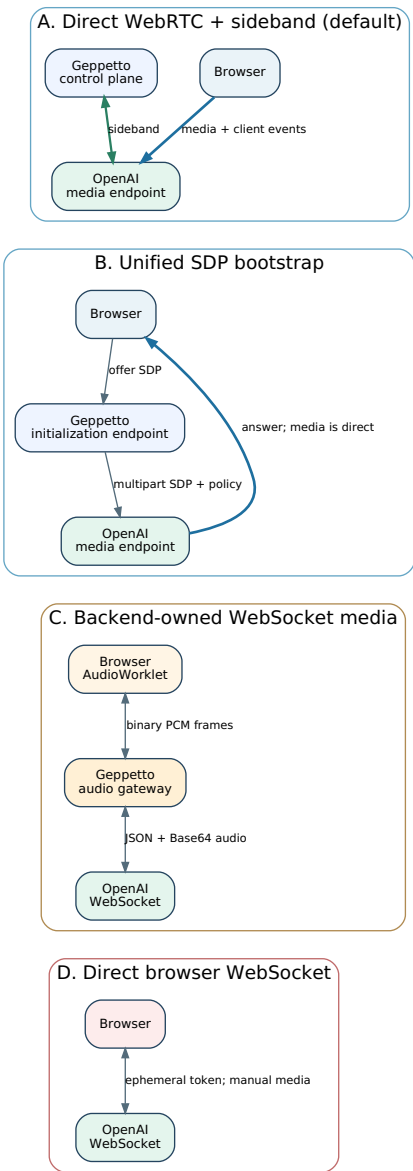
3.7 Events can be concurrent and out of order

Delta streams for audio, text, transcript, and function arguments can overlap. Completion events for different transcription turns are not guaranteed to arrive in order; the official transcription guide requires matching by `item_id`. [OAI-TRANSCRIPTION](#)

Therefore the adapter must index provider-native state by session/call/response/item/content IDs internally, while mapping only stable semantic events to Geppetto’s canonical IDs.

4. Browser-to-API transport study

Transport topologies and their operational consequences



4.1 Option A: direct browser WebRTC with ephemeral client secret

Flow

1. Browser authenticates to Geppetto.
2. Geppetto calls `POST /v1/realtime/client_secrets` using the permanent server key.
3. Geppetto returns the short-lived client secret and a Geppetto live-session identifier.
4. Browser creates `RTCPeerConnection`, adds microphone track and event data channel, creates an SDP offer, and POSTs the offer directly to OpenAI using the short-lived secret.
5. Browser installs the SDP answer and begins media exchange.
6. Browser extracts the OpenAI `call_id` from the `Location` response header and sends it to Geppetto.
7. Geppetto opens a sideband WebSocket to the same call.

Advantages

- Geppetto is not in the SDP forwarding path after minting the token.
- Audio never transits Geppetto.
- Browser media behavior is handled by WebRTC.
- Infrastructure cost and bandwidth at Geppetto remain low.

Risks and mitigations

- Client secrets are short-lived but can create multiple sessions until they expire. The API allows 10 to 7,200 seconds and defaults to 600 seconds. Use the smallest practical TTL, normally 30-60 seconds, bind issuance to an authenticated application session, rate-limit it, and record one logical issuance. [OAI-CLIENT-SECRETS](#)
- Attached client-secret configuration can be overridden by the client. Treat it as a default, not a security boundary. Keep privileged tools server-side and reauthorize every tool action.
- There is a sideband attach race. Mute/disable the microphone track until the backend acknowledges attachment, or do not advertise privileged tools until sideband control is active.
- A hostile client can use its protocol access to send session events. If the requirement is that the client must be technically unable to modify session configuration, use a backend-owned control/media connection rather than relying on UI restrictions.

Fit

Best when the application favors minimal bootstrap latency and can tolerate a two-step token/call attachment flow.

4.2 Option B: unified SDP interface through Geppetto

Flow

1. Browser creates a WebRTC SDP offer.
2. Browser POSTs the raw offer to an authenticated Geppetto endpoint.

3. Geppetto creates a multipart request containing fields `sdp` and `session`, then calls `/v1/realtime/calls` with the permanent key.
4. Geppetto captures the answer SDP and `Location/call_id`.
5. Geppetto can attach sideband before returning success.
6. Browser installs the answer; media then flows directly between browser and OpenAI.

OpenAI describes the unified interface as simpler, with the tradeoff that the application server is in the critical path for session initialization. [OAI-WEbrtc](#)

Advantages

- One browser bootstrap call.
- Server chooses the authoritative initial session configuration.
- `call_id` is available to the backend immediately.
- Sideband can attach before the browser enables microphone input.
- Easier correlation and audit of session creation.

Disadvantages

- Geppetto must proxy the SDP setup request and handle multipart requests correctly.
- The backend is a dependency for initial call setup.
- Client protocol access still exists after the WebRTC data channel opens; the server-provided initial config alone is not an authorization boundary.

Fit

Recommended default for tool-enabled production sessions because it closes the sideband attach race and simplifies policy/correlation, while still keeping media off the backend.

4.3 Option C: direct browser WebSocket to OpenAI

The official WebSocket guide shows that an ephemeral token can be used from browser-like environments, but explicitly recommends WebRTC for browser/mobile clients. [OAI-WEBsocket](#)

Why it is usually inferior

- Browser audio must be captured as samples, downmixed, resampled, converted to PCM16, Base64-encoded, and wrapped in JSON.
- Output audio must be decoded and scheduled without gaps.
- The browser must implement interruption truncation accurately.
- WebSocket does not supply the media-specific behavior provided by WebRTC.
- The client still requires a backend to mint an ephemeral secret.

When it can be justified

- A constrained runtime has WebSocket but not functional WebRTC.
- The application already owns a mature low-latency PCM engine.
- The product intentionally wants exact application-level media framing and accepts the complexity.

This should not be the first browser implementation.

4.4 Option D: browser -> Geppetto WebSocket -> OpenAI WebSocket

This topology places Geppetto in the media path.

Browser input pipeline

Use an `AudioWorklet`, not `MediaRecorder`, for the primary live PCM path:

1. Capture Float32 samples from the browser audio graph.
2. Downmix to one channel.
3. Resample from the device/AudioContext rate (commonly 44.1 or 48 kHz) to the configured 24 kHz PCM rate.
4. Convert to signed little-endian PCM16.
5. Batch into small binary frames with sequence number and capture timestamp.
6. Send binary frames to Geppetto.
7. Geppetto Base64-encodes the bytes and emits `input_audio_buffer.append` JSON to OpenAI.

`MediaRecorder` is useful for archival recordings, but it commonly emits compressed container chunks such as WebM/Opus rather than raw PCM. Using it for a low-latency Realtime WebSocket path requires decode/transcode and introduces additional buffering.

Output pipeline

1. Geppetto receives `response.output_audio.delta` Base64 data.
2. Decode once on the server or forward compact binary PCM to the browser.
3. Browser writes samples into an `AudioWorklet` ring buffer.
4. Track exact samples played, not merely received.
5. On `input_audio_buffer.speech_started`, stop queued playback and report the played duration.
6. Geppetto sends `conversation.item.truncate` with the corresponding `audio_end_ms`.

Bandwidth estimate

At 24,000 samples/s, mono PCM16 is:

24,000 samples/s * 2 bytes/sample	= 48,000 bytes/s
Base64 expansion	≈ 64,000 bytes/s
Raw bit rate	≈ 384 kbit/s
Base64 payload bit rate	≈ 512 kbit/s before JSON/TLS framing

This is manageable for one session, but expensive at scale compared with letting WebRTC manage compressed media. A 20 ms PCM frame contains 480 samples or 960 raw bytes; a 100 ms frame contains 2,400 samples or 4,800 raw bytes. A starting range of 40-80 ms balances overhead and latency, but must be benchmarked under realistic devices and networks.

Operational costs

- all input and output audio bandwidth crosses Geppetto;

- Base64 conversion and JSON allocation occur on the OpenAI leg;
- two independent real-time sockets must be backpressured;
- every backend deployment/incident can disrupt media;
- autoscaling must account for connection count, audio byte rate, and CPU;
- recording/privacy obligations attach directly to the backend.

Fit

Use when server media custody is a requirement, not merely because it seems architecturally familiar.

4.5 Option E: chained voice

Chained voice uses live speech-to-text, an existing text agent, and text-to-speech. It may use WebRTC for browser transcription, WebSocket for server audio, or a browser/backend hybrid.

It provides:

- explicit intermediate text;
- straightforward policy checks before generation or speech;
- reuse of existing `Session`, `Engine`, and `toolloop.Loop`;
- deterministic choice of when to speak;
- easier durable transcripts and approvals.

It usually has more latency and less natural barge-in than native speech-to-speech. OpenAI recommends the chained pattern for predictable workflows and extending an existing text agent.

[OAI-VOICE-AGENTS](#)

4.6 Decision matrix

Scores are relative for Geppetto's likely use cases: 5 is favorable, 1 is unfavorable.

Topology	Browser latency/ media behavior	Backend load	Server policy/tool control	Implementation complexity	Recommend ed use
Direct WebRTC + unified bootstrap + sideband	5	5	4	3	Default browser voice agent
Direct WebRTC + ephemeral secret + sideband	5	5	4	3	Lower bootstrap coupling; manage attach race
Direct browser WebSocket	2	5	2	1	Exceptional runtime constraints
Backend media proxy	3	1	5	1	Compliance, telephony, recording, transcoding
Chained voice	3	3	5	3	Approval-heavy, auditable, existing text workflows
Pure browser with permanent key	1	5	1	4	Reject: credential exposure

5. Recommended target architecture

5.1 Separate four responsibilities

Live domain layer

Provider-neutral session state, commands, snapshots, terminal errors, and lifecycle. It must not know about WebRTC browser APIs.

OpenAI Realtime adapter

Exact provider protocol types, client-secret/unified bootstrap calls, server WebSocket transport, sideband attachment, event reducer, tool mapping, and provider diagnostics.

Browser media client

`RTCPeerConnection`, device permissions, microphone controls, autoplay/user gesture, remote audio, data-channel events, call bootstrap, and browser metrics.

Existing Geppetto services

Session IDs, canonical events, tool registry/executor, settings loading, persistence, logs/traces/ metrics, and Goja integration.

5.2 Media plane versus semantic plane

A central rule is to keep media bytes separate from semantic state.

Plane	Examples	Handling
Media	microphone samples, remote audio packets, output PCM deltas	WebRTC track or dedicated bounded audio transport
Provider protocol	<code>session.updated</code> , item IDs, audio deltas, argument deltas	typed adapter + reducer
Reliable control	tool output, truncate, cancel, session update	bounded command queue; transport write pump
Canonical semantic events	transcript delta, response started, tool call, error	<code>events.EventSink</code>
Durable state	final transcripts, completed tool result, completed/truncated assistant item	Turn/session persistence at stable barriers

The default `EventSink` should never carry raw PCM or Base64 audio. Doing so would multiply copies, encourage accidental logging, and let slow telemetry subscribers interfere with media.

5.3 Why a new live subsystem is preferable to extending Engine

A parallel subsystem preserves backward compatibility and makes invariants explicit:

Finite inference ----- one input Turn one output Turn method completion is success boundary SSE can be reduced internally finite tool loop invokes engine repeatedly <code>ExecutionHandle.Wait()</code> returns result	Live session ----- mutable session configuration many conversation Items/Responses close/failure is session boundary independent duplex event stream tools occur inside the live connection <code>Session.Done()</code> signals terminal state
--	---

Adapters can share lower-level utilities, but their public contracts should not be merged.

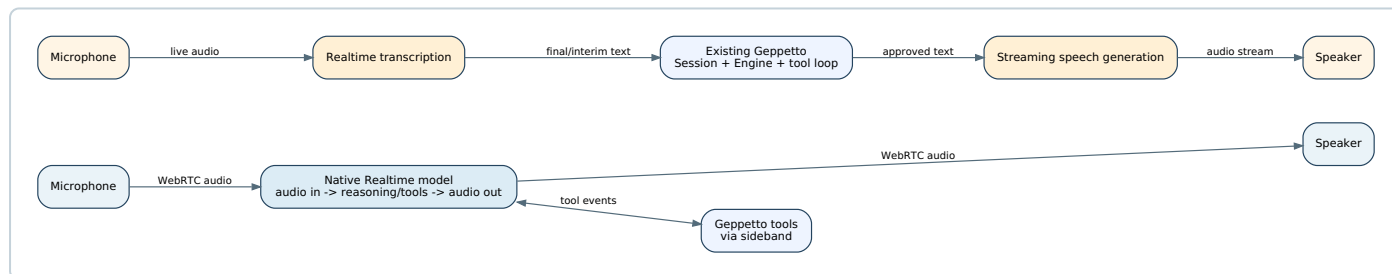
5.4 Provider call identity

Use two identifiers:

- **Geppetto live SessionID:** stable across the user-facing live interaction and used for application correlation.
- **OpenAI call/session IDs:** provider-native capabilities and provenance kept inside adapter state/ debug metadata.

A reconnect or 60-minute rollover may create a new provider call while retaining the same higher-level Geppetto session or creating a linked successor, depending on persistence policy.

6. Supporting both native and chained voice



The phrase “both designs” has two useful interpretations in this codebase, and both should be adopted.

6.1 Both existing Geppetto interaction designs

Session-centered design

Use the session concept for:

- stable identity;
- lifecycle and close/cancel semantics;
- append-only stable snapshots;
- fork/resume at transcript barriers;
- persistence and UI ownership.

Streaming event design

Use EventSink/EventEmitter for:

- incremental text/transcript updates;
- live state changes;
- tool progress;
- response lifecycle;
- rate limits and errors.

The live session owns correctness; the event layer observes it. This preserves the current event system’s best-effort contract.

6.2 Both voice architectures

Mode A: native_realtime

Use direct audio input/output in one Realtime model session.

Best for:

- conversational assistants;

- low first-audio latency;
- natural turn-taking;
- barge-in;
- voice style/prosody;
- real-time tool interactions.

Tradeoffs:

- less explicit control over intermediate text;
- input transcript is an asynchronous auxiliary signal;
- client and server must reason about ongoing provider conversation state;
- strict approval before every spoken phrase is harder.

Mode B: chained

Use live transcription -> current Geppetto text workflow -> streaming TTS.

Best for:

- support and transaction flows;
- approvals or human review;
- deterministic policy checkpoints;
- durable, authoritative text records;
- reuse of existing prompts, engines, tools, and middleware;
- providers that do not offer native speech-to-speech.

Tradeoffs:

- greater end-to-end latency;
- more orchestration components;
- barge-in and prosody require explicit implementation;
- the spoken result may feel less fluid.

6.3 Common product facade, separate internals

A product-facing browser API can use a shared vocabulary:

```
const voice = await client.voiceSession({
  mode: "native_realtime", // or "chained"
  transport: "webrtc_direct",
  profile: "support-default",
});

voice.on("input-transcript-delta", renderUserCaption);
voice.on("output-transcript-delta", renderAssistantCaption);
voice.on("tool-call-started", renderToolStatus);
await voice.connect();
```

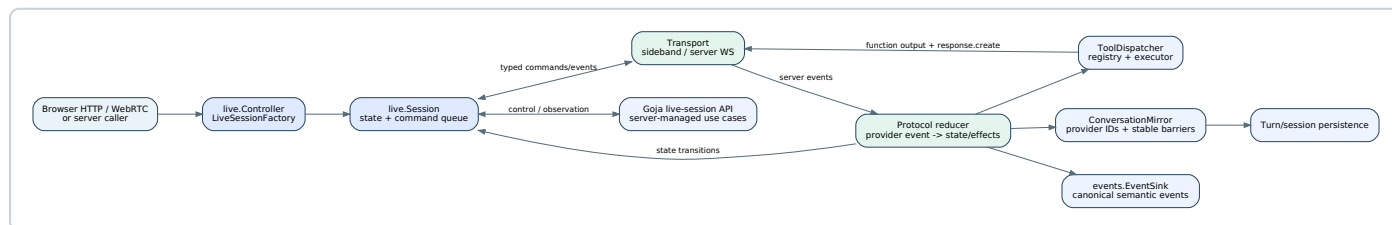
Internally, the modes should remain distinct:

- native mode opens `live.Session` and the OpenAI Realtime adapter;

- chained mode opens a transcription session, commits a text Turn into the existing `Session`, runs the normal engine/tool loop, and streams TTS output.

Do not make native Realtime pretend that every VAD boundary is `session.next()`. Provider Conversation Items and Responses are finer-grained and interruption-sensitive.

7. Detailed Go package and interface design



7.1 Proposed package layout

```
pkg/inference/live/
types.go           # IDs, states, snapshots
config.go          # provider-neutral live config
command.go         # reliable commands
session.go         # lifecycle, queues, Done/Err/Close
controller.go      # open and attach operations
transport.go       # provider-neutral duplex transport
reducer.go         # generic reducer contracts/effects
mirror.go          # stable semantic conversation state
tool_dispatcher.go # reliable tool loop integration
persistence.go     # stable barrier interface
errors.go

pkg/steps/ai/openai_realtime/
settings.go
client_secret.go
unified_sdp.go
websocket_transport.go
sideband.go
protocol_client_events.go
protocol_server_events.go
reducer.go
event_mapper.go
tool_mapper.go
observability.go
testdata/*.jsonl

pkg/events/live_events.go
pkg/js/modules/geppetto/api_live_session.go
pkg/js/modules/geppetto/api_live_event_payloads.go
```

The uploaded repository does not contain a browser UI/media package. Browser TypeScript should therefore be a distinct deliverable or downstream package, for example:

```
web/realtime/
voice_session.ts
openai_webrtc_transport.ts
audio_devices.ts
```

```
event_types.ts
browser_metrics.ts
```

7.2 Core lifecycle interfaces

```
package live

type State string

const (
    StateNew          State = "new"
    StateConnecting   State = "connecting"
    StateActive       State = "active"
    StateClosing      State = "closing"
    StateClosed       State = "closed"
    StateFailed       State = "failed"
)

type Session interface {
    ID() string
    State() State
    Send(ctx context.Context, cmd Command) error
    Snapshot() Snapshot
    Done() <-chan struct{}
    Err() error
    Close(ctx context.Context) error
}

type Controller interface {
    OpenServerSession(ctx context.Context, cfg SessionConfig) (Session, error)
    AttachSideband(ctx context.Context, callID string, cfg AttachConfig) (Session, error)
}

type BrowserBootstrapService interface {
    CreateClientSecret(
        ctx context.Context,
        req BootstrapRequest,
    ) (*ClientSecretBootstrap, error)

    AnswerWebRTC(
        ctx context.Context,
        req WebRTCOfferRequest,
    ) (*WebRTCAnswer, error)
}
```

`Session.Send` is for low-rate reliable commands. It is not for raw WebRTC media.

7.3 Transport interface

```
type Transport interface {
    Send(ctx context.Context, event ClientEvent) error
    Receive(ctx context.Context) (ServerEvent, error)
    Close(ctx context.Context) error
}
```

Implementation rules:

- exactly one read pump owns `Receive`;
- exactly one write pump serializes protocol writes;

- callers submit to a bounded command queue;
- terminal error is recorded once;
- context cancellation stops pumps and tool work;
- callbacks/events are never invoked while internal locks are held;
- oversized provider events and audio payloads are bounded before logging.

For direct browser WebRTC, Geppetto's `Transport` is the sideband WebSocket, not the browser media connection. Browser control events can be mirrored or observed through the same OpenAI call.

7.4 Commands

Start with explicit typed commands:

```
type Command interface { isLiveCommand() }

type UpdateSession struct { Patch SessionPatch }
type CreateConversationItem struct { Item ItemInput }
type CreateResponse struct { Response ResponseRequest }
type CancelResponse struct { ResponseID string }
type CommitInputAudio struct{}
type ClearInputAudio struct{}
type AppendInputAudio struct { PCM []byte } // server WebSocket only
type TruncateItem struct {
    ProviderItemID string
    ContentIndex   int
    AudioEndMS     int64
}
type SendToolOutput struct {
    CallID string
    JSON   string
}
type Close struct { Reason string }
```

Avoid a generic `SendJSON(any)` in the public live package. The adapter can expose an escape hatch under an explicitly unstable/debug namespace, but normal code should be compile-time checked.

7.5 Provider protocol reducer

The OpenAI adapter should decode into exact event structs and reduce them through a deterministic state machine:

```
type Reduction struct {
    NewState ProviderState
    Events   []events.Event
    Effects  []Effect
    Barriers []PersistenceBarrier
}

func Reduce(state ProviderState, event ServerEvent) (Reduction, error)
```

Effects can include:

- enqueue tool execution;
- send tool output;

- send `response.create` ;
- persist final transcript;
- mark response cancelled;
- record usage/rate limit;
- close or fail session.

Pure reducer tests can replay JSONL fixtures without network access.

7.6 Session state machine

```
New -> Connecting -> Active -> Closing -> Closed
      \-> Failed
Active -----> Failed
Closing -----> Failed
```

State transitions must be monotonic. `Close` is idempotent. `Err()` returns nil for an orderly close and a terminal cause for failure. A failed sideband does not necessarily stop browser audio immediately; policy should decide whether to fail closed, attempt reattach, or allow a tool-free degraded mode.

Recommended default:

- if privileged tools are advertised, sideband loss fails closed and the browser is instructed to end the call;
- if the session is explicitly tool-free, sideband loss may degrade to local browser operation while emitting a high-severity event.

7.7 Live settings

Add a separate settings block rather than overloading `Chat.Stream` :

```
type RealtimeSettings struct {
    Model          string
    Voice          string
    OutputModalities []string
    Transport      string // webrtc_direct, server_websocket
    BootstrapMode  string // unified_sdp, client_secret
    SidebandRequired bool

    InputFormat      AudioFormat
    OutputFormat     AudioFormat
    InputTranscription *TranscriptionSettings
    NoiseReduction   *NoiseReductionSettings
    TurnDetection    TurnDetectionSettings

    ReasoningEffort string
    MaxOutputTokens int
    Truncation      TruncationSettings
    Include         []string

    ClientSecretTTL time.Duration
    MaxSessionAge  time.Duration
    RolloverLead    time.Duration
    Profile         string
}
```

Configuration rules:

- model and voice are selected by profile;
- validate transport-specific fields;
- validate PCM rates against provider support;
- reject client-secret TTL outside provider limits;
- require sideband when privileged tools are present;
- lock voice selection before first audio output;
- do not hard-code `gpt-realtime-2.1`; current docs identify `gpt-realtime-2` as the strongest reasoning voice family while examples use point releases. [OAI-REALTIME-MODELS](#) [OAI-WEBRTC](#)

7.8 Factory integration

```
type LiveSessionFactory interface {
    BuildController(
        ctx context.Context,
        settings *settings.InferenceSettings,
        deps Dependencies,
    ) (live.Controller, error)
}
```

`engine/factory` remains unchanged except for shared dependency helpers. A higher-level `AgentFactory` may expose both finite and live construction, but their return types remain distinct.

7.9 Official SDK versus protocol types

The current official Go SDK exposes Realtime REST/session/client-secret/call types. Geppetto can use it for authenticated REST operations if it fits dependency policy. The live WebSocket pump should still remain behind Geppetto's `Transport` interface so protocol handling, reconnect behavior, tests, and logging are under project control. Avoid leaking SDK structs across the provider-neutral `live` package.

8. Browser client design

8.1 Browser WebRTC responsibilities

The browser package should own:

- user-gesture-gated microphone permission;
- input/output device selection;
- `RTCPeerConnection` lifecycle;
- microphone track enable/disable;
- remote audio element/stream;
- data-channel serialization and event dispatch;
- browser connection state and metrics;
- sideband-ready gating;

- accessibility captions from transcript events;
- explicit close and cleanup.

It should not own privileged tool implementations or permanent credentials.

8.2 Recommended unified-bootstrap sketch

```
type BootstrapAnswer = {
  sdp: string;
  geppettoSessionId: string;
  callId?: string; // backend may keep this opaque
};

export async function connectRealtimeVoice(): Promise<RTCPeerConnection> {
  // Call from a user click so microphone and audio playback are allowed.
  const pc = new RTCPeerConnection();
  const dc = pc.createDataChannel("oai-events");

  const output = document.createElement("audio");
  output.autoplay = true;
  output.playsInline = true;
  pc.ontrack = (event) => {
    output.srcObject = event.streams[0];
  };

  const media = await navigator.mediaDevices.getUserMedia({
    audio: {
      echoCancellation: true,
      noiseSuppression: true,
      autoGainControl: true,
      channelCount: 1,
    },
  });

  const track = media.getAudioTracks()[0];
  track.enabled = false; // enable only after sideband-ready response
  pc.addTrack(track, media);

  dc.addEventListener("message", (event) => {
    const message = JSON.parse(String(event.data));
    dispatchRealtimeEvent(message);
  });

  const offer = await pc.createOffer();
  await pc.setLocalDescription(offer);

  const response = await fetch("/api/live/openai/webrtc", {
    method: "POST",
    credentials: "include",
    headers: {
      "Content-Type": "application/sdp",
      "X-CSRF-Token": readCsrfToken(),
    },
    body: offer.sdp,
  });
  if (!response.ok) throw new Error(`bootstrap failed: ${response.status}`);

  const answer = await response.json() as BootstrapAnswer;
  await pc.setRemoteDescription({ type: "answer", sdp: answer.sdp });

  await waitForDataChannelOpen(dc);
  await waitForSidebandReady(answer.geppettoSessionId);
  track.enabled = true;
}
```

```
    return pc;
}
```

The actual response can return raw `application/sdp` plus metadata headers instead of JSON. JSON is shown to make correlation explicit; either contract is acceptable if carefully specified.

8.3 Ephemeral-secret sketch

```
const tokenResponse = await fetch("/api/live/openai/client-secret", {
  method: "POST",
  credentials: "include",
  headers: { "X-CSRF-Token": readCsrfToken() },
});
const bootstrap = await tokenResponse.json();

const sdpResponse = await fetch("https://api.openai.com/v1/realtime/calls", {
  method: "POST",
  body: offer.sdp,
  headers: {
    Authorization: `Bearer ${bootstrap.clientSecret}`,
    "Content-Type": "application/sdp",
  },
});

const location = sdpResponse.headers.get("Location");
const callId = location?.split("/").pop();
if (!callId) throw new Error("missing realtime call id");

await fetch(`/api/live/${bootstrap.geppettoSessionId}/attach`, {
  method: "POST",
  credentials: "include",
  headers: {
    "Content-Type": "application/json",
    "X-CSRF-Token": readCsrfToken(),
  },
  body: JSON.stringify({ callId }),
});
```

Keep the microphone disabled until attach acknowledgement. Never log the client secret or put it in a URL/query string.

8.4 Browser event surface

Expose provider-neutral events, while allowing an optional raw debug stream:

```
type VoiceEvents = {
  "state-changed": { from: VoiceState; to: VoiceState };
  "input-speech-started": { at: number };
  "input-speech-stopped": { at: number };
  "input-transcript-delta": { itemId: string; delta: string };
  "input-transcript-finished": { itemId: string; text: string };
  "output-transcript-delta": { responseId: string; delta: string };
  "output-transcript-finished": { responseId: string; text: string };
  "tool-call-started": ToolCallView;
  "tool-call-finished": ToolResultView;
  "interrupted": { responseId?: string };
  "error": { code: string; message: string; recoverable: boolean };
};
```

Raw OpenAI events may be exposed only under `debug.onProviderEvent` with redaction and size limits.

8.5 Device and browser edge cases

- Begin connection from a user gesture to satisfy microphone and autoplay restrictions.
- Handle permission denial distinctly from network failure.
- Listen for `devicechange`; do not silently switch microphones during a sensitive call.
- Stop all media tracks and close the data channel/peer connection on teardown.
- Surface `connectionState`, `iceConnectionState`, and data-channel state.
- Treat page suspension/background throttling as a possible call-quality event.
- Avoid rendering every transcript delta through a large framework tree; batch UI updates at animation-frame cadence.
- Test browser echo cancellation together with provider noise reduction. More processing is not always better.

8.6 Browser audio proxy implementation details

For the optional backend proxy:

- capture and playback in `AudioWorkletProcessor` instances;
- use bounded ring buffers;
- use `WebSocket.bufferedAmount` thresholds;
- send sequence number, sample count, and capture timestamp in a compact binary header;
- resample using a stateful filter, not independent per-frame naive interpolation;
- do not spawn an `AudioBufferSourceNode` for every small output chunk;
- track **played samples** for interruption, not queued samples;
- if `SharedArrayBuffer` is used, configure cross-origin isolation deliberately; otherwise use transferable buffers and account for copying.

9. Backend endpoint and sideband design

9.1 Endpoint set

A minimal production API can expose:

```
POST /api/live/openai/webrtc
Request: application/sdp
Response: answer SDP + Geppetto live session metadata
Role: unified SDP bootstrap; server captures call_id and attaches sideband

POST /api/live/openai/client-secret
Request: authenticated application request
Response: short-lived client secret + Geppetto live session ID
Role: alternative ephemeral bootstrap

POST /api/live/{sessionID}/attach
Request: call_id obtained by browser
Role: attach sideband for ephemeral flow
```

```
GET /api/live/{sessionID}/status
  Role:      sideband-ready and state checks

POST /api/live/{sessionID}/close
  Role:      explicit user/application termination
```

The first production slice can implement only unified bootstrap, status, and close.

9.2 Unified endpoint sequence

Browser	Geppetto	OpenAI
create offer		
POST SDP ----->		
	POST multipart sdp+session>	
	<-- answer + Location -----	
	connect sideband(call_id) --> WebSocket	
	<-- session.created/updated --	
<-- answer + ready --		
set remote SDP		
enable microphone		
===== WebRTC media and events =====>		

The endpoint should not return “ready” until:

- OpenAI returned a successful SDP answer;
- a valid `call_id` was captured;
- sideband connected if required;
- initial server-controlled `session.update` was acknowledged or timed out according to policy;
- tool definitions were installed if tools are enabled.

9.3 Sideband session ownership

The backend stores a short-lived live-session record:

```
type LiveRecord struct {
  SessionID   string
  UserID      string
  Provider    string
  ProviderCallID string // secret/capability-like; encrypt or protect
  State       live.State
  Profile     string
  CreatedAt   time.Time
  SidebandReadyAt *time.Time
  ExpiresAt   time.Time
}
```

Do not expose the provider `call_id` unless the browser flow requires it. Treat it as a capability for attaching to the live call.

9.4 Sideband read loop

The sideband loop should:

1. decode a bounded JSON event;
2. attach receive timestamp and provider sequence/debug IDs;

3. reduce into provider state;
4. synchronously enqueue required protocol effects;
5. enqueue tool work on a bounded worker path;
6. emit canonical best-effort events;
7. write persistence barriers asynchronously with ordering per live session;
8. update metrics.

Tool output must be serialized through the single write pump. Event subscribers must never write directly to the WebSocket.

9.5 Sideband loss policy

Classify failures:

- **transient network failure before tools are advertised:** retry attach with jitter within a short deadline;
- **loss during a tool-free session:** optional degraded continuation, with clear UI warning;
- **loss while privileged tools are available or running:** fail closed, cancel/close the call, and prevent further tool output;
- **provider reports terminal error:** persist final stable state and close;
- **browser disconnects while sideband remains:** close after a short grace period unless a handoff flow exists.

OpenAI does not turn sideband into an application authorization boundary. Tool handlers remain responsible for user/account authorization.

9.6 Go HTTP implementation notes

- Limit SDP request size.
- Enforce `Content-Type` and reject unexpected multipart/browser payloads.
- Set upstream timeouts for OpenAI REST bootstrap.
- Do not retry a successful call creation blindly; retries can create duplicate live calls.
- Generate an idempotency key for application bootstrap and cache the result briefly.
- Propagate a stable, privacy-preserving safety identifier from the trusted backend as documented by OpenAI. [OAI-WEBRTC](#)
- Redact Authorization, client secrets, SDP, and call IDs from logs.
- Apply the repository's existing outbound URL/local-network safety policies if base URLs are configurable.

10. Events, correlation, tools, and persistence

10.1 Canonical event additions

Add event types in `pkg/events/live_events.go`:

```
live-session-started
live-session-updated
live-session-ended
live-session-failed

response-started
response-finished
response-cancelled
conversation-item-added
conversation-item-finished

audio-input-started
audio-input-stopped
audio-input-committed
input-transcript-delta
input-transcript-finished
audio-output-started
audio-output-stopped
output-transcript-delta
output-transcript-finished

rate-limits-updated
```

Reuse existing `text-delta`, `text-segment`, `error`, `interrupt`, and `tool` events when their semantics match. Do not duplicate all provider event names one-for-one in the canonical layer.

10.2 Correlation mapping

OpenAI concept	Geppetto mapping
one live call/session	canonical <code>SessionID</code> ; provider call ID in adapter state
one model Response	<code>ProviderCallID</code> or a response-scoped canonical ID
output content/transcript stream	<code>SegmentID</code>
function <code>call_id</code>	<code>ToolCallID</code>
provider event ID	debug metadata used to correlate errors
provider item ID	adapter mirror key and provenance metadata
stable user/model exchange	a persisted Turn snapshot

Provider item IDs are essential internally because deltas and completions can arrive out of order. They should not replace canonical IDs in public routing.

10.3 Reliable tool dispatcher

```
type ToolDispatcher struct {
    Registry tools.ToolRegistry
    Executor tools.ToolExecutor
    Send      func(context.Context, live.Command) error
    Seen      IdempotencyStore // keyed by provider call_id
}
```

Processing steps:

1. Assemble function arguments by provider item/call ID.
2. Validate final JSON against the registered tool schema.
3. Deduplicate the provider `call_id`.
4. Reauthorize the tool against the current application user/session.
5. Apply deadline, cancellation, and confirmation policy.
6. Execute through the existing executor.
7. Emit existing canonical tool lifecycle events.
8. Send `function_call_output` with the same `call_id`.
9. Send `response.create` if the provider does not automatically continue.
10. Persist a stable tool-call/tool-result barrier.

The dispatcher must not subscribe to `EventSink` to discover calls; it receives a reliable reducer effect.

10.4 Client-side versus server-side tools

Default rule:

- privileged, data-bearing, or state-changing tools execute on Geppetto;
- harmless UI-only actions may be represented as client events, but should not be advertised as trusted tools unless the backend can verify their result;
- write tools require explicit authorization and, where appropriate, user confirmation;
- tool output is untrusted data until validated and escaped before prompt insertion.

Direct WebRTC should not move tool credentials or internal APIs into the browser.

10.5 Conversation mirror

During a live call, maintain a mutable provider mirror:

```
type ConversationMirror struct {
    Session      SessionSummary
    Items        map[string]*ItemState
    ItemOrder    []string
    Responses    map[string]*ResponseState
    ActiveAudio  *PlaybackState
    Usage        UsageState
}
```

The mirror tracks partial deltas, but persistence only occurs at barriers.

10.6 Persistence barriers

Recommended stable barriers:

1. user audio is committed and final input transcript arrives;
2. assistant content item is done, or has been correctly truncated after interruption;
3. function call and function output are complete;
4. response is done/cancelled and semantic state is stable;
5. session closes or rolls over.

Do not persist:

- every audio delta;
- every transcript character delta;
- unplayed assistant transcript after interruption;
- provider event objects wholesale;
- secrets, SDP, or raw call IDs in ordinary Turn metadata.

10.7 Mapping to Turns

Two reasonable policies exist.

Exchange-based snapshots

Append one new Turn after a completed user speech turn and the corresponding assistant response/tool cycle. This is closest to current text sessions and easiest for replay.

Item-based snapshots

Append after each completed meaningful Conversation Item. This captures multi-part and tool-heavy flows more precisely but creates more Turn snapshots.

Recommendation: start with exchange-based snapshots and include item provenance metadata. Use item-based persistence only where audits require it.

10.8 Interrupted assistant output

WebRTC/SIP automatically truncate unplayed output on user interruption. WebSocket clients must stop playback, track played duration, and send `conversation.item.truncate`. [OAI-CONVERSATIONS](#)

Persistence must use the post-truncation semantic state. Otherwise Geppetto will store words the user never heard and may replay them as conversation context later.

10.9 Transcript source metadata

```
type TranscriptMeta struct {
    Source          string // input_async, output_audio, output_text, chained_stt
    ProviderModel   string
    ProviderItemID  string
    Final           bool
    Interrupted     bool
}
```



```
StartMS      int64
EndMS        int64
ConfidenceClass string // optional provider-neutral bucket, not invented score
}
```

Do not manufacture confidence values when the provider does not supply them.

11. Security and privacy model

11.1 Security boundaries

Asset or action	Trusted boundary
permanent OpenAI API key	Geppetto backend only
short-lived client secret	authenticated browser memory, never persistent storage
provider call ID	backend capability record; browser only when required
tool credentials	backend tool executor only
authorization to change external state	backend application policy, not model instructions
raw microphone audio	browser/OpenAI by default; backend only in explicit proxy/record mode
transcripts	classified application data with explicit retention policy

11.2 Client-secret handling

Official client secrets:

- are short-lived;
- can have a 10-7,200 second TTL;
- default to 600 seconds;
- can create multiple sessions until expiry;
- can carry session configuration that the client may override. [OAI-CLIENT-SECRETS](#)

Application policy should therefore:

- use 30-60 seconds unless measurements require longer;
- issue only after user/session authorization;
- enforce per-user and per-IP rate/concurrency quotas;
- bind the issuance record to a Geppetto SessionID and expected profile;
- never store the token in localStorage, cookies, URLs, analytics, or logs;
- reject reuse at the application layer where observable, while acknowledging the provider token itself is not guaranteed single-use;
- assume a malicious browser can alter provider session events.

11.3 Session configuration is not authorization

Prompts, tool descriptions, `session.update`, and client-secret defaults influence model behavior; they do not authorize access to databases, purchases, email, files, or other state changes.

Every tool invocation must independently establish:

- authenticated user identity;
- tenant/account scope;
- permission for the requested operation;
- validation of arguments;
- confirmation requirements;
- idempotency and replay protection;
- output filtering.

If policy requires preventing the browser from issuing any provider control event, direct WebRTC is not a hard isolation mechanism because the connected client owns a data channel and credential. Use a backend-owned WebSocket/media gateway for that threat model.

11.4 Bootstrap endpoint defenses

- authenticated session required;
- CSRF protection for cookie-authenticated requests;
- exact Origin/Host checks;
- restrictive CORS;
- request-body and SDP size limits;
- content-type validation;
- rate limit by user, tenant, IP, and device fingerprint where lawful;
- maximum concurrent live sessions;
- spend/quota checks before call creation;
- no open redirect or user-controlled OpenAI base URL;
- upstream timeout and circuit breaker;
- idempotency record to avoid duplicate calls;
- audit event without storing secrets or SDP.

11.5 Audio and transcript privacy

Default privacy posture:

- direct WebRTC, so Geppetto does not receive raw audio;
- no recording;
- final transcript persistence only where product behavior requires it;
- transcript redaction/classification before logs or analytics;
- no raw provider event logging in production by default;
- explicit user notice and consent if recording is added;

- separate retention policy for media, transcript, tool data, and metrics;
- deletion workflow that covers external media objects and derived transcripts.

A request to “record calls” materially changes the architecture. Direct WebRTC does not automatically give the backend an audio copy. Recording would require explicit browser capture/upload, a parallel media path, or the backend proxy topology.

11.6 Logging rules

Never log:

- Authorization headers;
- client secrets;
- complete SDP offers/answers;
- raw `call_id` values at normal log levels;
- Base64 audio;
- full tool arguments/results unless a data classification policy allows it;
- full transcripts by default.

Log bounded metadata:

- hashed/stable application user identifier;
- canonical SessionID;
- provider/model/profile;
- event type and byte size;
- latency and status;
- redacted error code;
- usage totals;
- tool name and outcome class.

11.7 Safety identifier

OpenAI recommends sending a stable, privacy-preserving safety identifier from the trusted backend. For client-secret flows, bind it when minting the secret; for unified calls and server WebSockets, set it in the backend request. [OAI-WEBRTC](#) [OAI-WEBSOCKET](#)

12. Reliability, interruption, and long-session behavior

12.1 Bounded queues and backpressure

Every live queue must have a bound and an overflow policy:

Queue	Overflow policy
reliable commands	block with context/deadline, then fail session if correctness cannot be preserved
provider read events	process continuously; fail on sustained inability to reduce
canonical EventSink	best-effort/drop according to existing sink policy, with drop metric
persistence writes	bounded per-session ordered queue; fail/degrade based on durability contract
browser proxy input audio	stop/pause capture or terminate; do not grow unbounded
browser proxy output audio	cap latency buffer; cancel/truncate rather than play stale speech

Do not solve backpressure by allowing unbounded channels.

12.2 Interruption behavior

WebRTC

The service manages output buffering and automatically truncates unplayed audio when VAD detects an interruption. Geppetto still records a canonical interrupt/cancel event and persists the resulting truncated content. [OAI-CONVERSATIONS](#)

WebSocket proxy

The application owns playback. On speech start:

1. stop output immediately;
2. cancel queued samples;
3. calculate played milliseconds from sample count;
4. identify the current provider item/content index;
5. send `conversation.item.truncate`;
6. wait for item/response state to stabilize;
7. persist only the heard portion.

This behavior requires deterministic integration tests because a small bookkeeping error corrupts future conversation state.

12.3 Session rollover

Current Realtime sessions have a 60-minute maximum. [OAI-CONVERSATIONS](#)

Set an application maximum below that limit, for example 55 minutes, and start graceful rollover with a lead time. A rollover procedure can:

1. prevent new long-running tool work;
2. wait for or cancel the active response;
3. persist final stable items;
4. create a structured summary/current-state block;

5. open a successor provider call;
6. seed authoritative context and tool state;
7. atomically switch the browser or instruct reconnect;
8. link predecessor/successor provider IDs under the Geppetto session.

Do not replay partial audio. Replay only stable transcript/tool context or a structured summary.

12.4 Reconnect semantics

A dropped WebRTC connection normally results in a new provider session rather than transparent restoration of the exact media stream. Define reconnect as:

- a new provider call;
- same or successor Geppetto SessionID according to product policy;
- restored stable conversation summary/items;
- no restoration of partial response/audio buffers;
- idempotency protection for tools that may have completed near disconnection.

12.5 Sideband attach race

For ephemeral bootstrap, the browser can establish the call before Geppetto attaches. Mitigations in order of preference:

1. use unified bootstrap and attach before returning answer;
2. keep the microphone track disabled until backend ready;
3. start with no privileged tools and add them only after sideband ready;
4. treat early client events as untrusted and close if policy was not installed.

12.6 Event ordering

- index by provider item/response IDs;
- accept transcript completion out of order across different items;
- require monotonically applied content indices per item where the protocol guarantees them;
- deduplicate event IDs when present;
- preserve receive order in debug traces without assuming it equals semantic order;
- make reducer operations idempotent where possible.

12.7 Tool idempotency

Provider retries, reconnect races, or local retries can repeat a function-call signal. Use the provider `call_id` plus Geppetto SessionID as an idempotency key. Store terminal tool outcome long enough to safely return the same output rather than executing a state-changing tool twice.

12.8 VAD tuning

Start with `server_vad` for predictable behavior. Evaluate `semantic_vad` for conversational use cases. Provide push-to-talk/manual mode as a fallback for noisy environments, domain-specific pauses, and accessibility needs.

Measure:

- false starts;
- clipped initial phonemes;
- premature turn endings;
- user-to-first-audio latency;
- interruption success;
- performance with headsets, laptop microphones, conference rooms, accents, and background audio.

13. Observability, capacity, and cost

13.1 Metrics

Bootstrap and connection

- `live_bootstrap_requests_total{mode,status}`
- `live_bootstrap_duration_seconds{mode}`
- `live_webrtc_connected_total`
- `live_sideband_attach_duration_seconds`
- `live_sideband_failures_total{reason}`
- `live_active_sessions{provider,profile,transport}`

Conversation quality

- end-of-user-turn to first output audio;
- speech-start and speech-stop detection latency;
- response duration;
- interruption count and truncation latency;
- reconnect and rollover counts;
- transcript finalization latency;
- browser playback buffer depth for proxy mode.

Tools

- tool call count by name/outcome;
- queue delay;
- execution latency;
- timeout/cancellation;
- duplicate call detection;

- confirmation denial.

Capacity

- sideband WebSocket count;
- read/write event rates;
- event and command queue depth;
- dropped best-effort events;
- proxy audio bytes/s and Base64 CPU if enabled;
- goroutine count and memory per session.

13.2 Tracing

Create one trace per live session with spans for:

- bootstrap;
- OpenAI call creation;
- sideband attach;
- each provider Response;
- each tool call;
- persistence barrier;
- rollover/reconnect;
- close/failure.

Do not create a span for every 20 ms audio frame. Aggregate media statistics.

13.3 Browser telemetry

Collect bounded WebRTC statistics at a low cadence, such as every 5-10 seconds, and summarize:

- connection/ICE state;
- round-trip time where available;
- inbound/outbound packet loss and jitter;
- audio level availability;
- concealed samples or jitter-buffer delay where available;
- device type category without storing raw device labels unless necessary.

Respect privacy and avoid high-cardinality labels.

13.4 Cost model

OpenAI documents two billing patterns:

- conversational Realtime responses accrue text/audio/image tokens, with later responses including growing conversation context;
- streaming transcription/translation sessions are billed by audio duration. [OAI-COSTS](#)

Practical controls:

- maximum session age;
- per-user concurrent-session quota;
- profile-specific model and reasoning effort;
- silence filtering/VAD;
- response length constraints;
- periodic context compaction or structured summary;
- early close when the page is hidden/disconnected;
- usage accounting from response usage/rate-limit events;
- budget alerts and kill switches.

Do not embed current prices in code. Fetch or configure them outside the protocol implementation because prices and model names change.

13.5 Context growth

Each conversational response sees the conversation state, so later turns can cost more. [OAI-COSTS](#) Current model guidance also emphasizes structured state for long sessions. [OAI-REALTIME-MODELS](#)

Geppetto should maintain:

- current task;
- authoritative current state;
- compact tool results;
- verified user/account facts;
- a summary of older dialogue;
- explicit stale/ superseded markers.

Avoid repeatedly injecting entire raw transcripts or verbose tool payloads.

14. Testing strategy

14.1 Protocol fixture tests

Store redacted JSONL event streams in `openai_realtime/testdata`:

- simple audio turn;
- text-only response;
- function call and result;
- argument deltas;
- user interruption;
- response cancellation;
- out-of-order transcript completions;
- rate-limit update;

- provider error linked to client `event_id`;
- sideband attach and session update;
- session close/timeout.

Replay each stream through the pure reducer and assert:

- final state;
- canonical event sequence;
- reliable effects;
- persistence barriers;
- no duplicate tools.

14.2 Fake transport tests

Implement a scripted transport:

```
type FakeTransport struct {
    Incoming chan ServerEvent
    Sent     []ClientEvent
    FailRead error
    FailSend error
}
```

Use it for lifecycle, cancellation, bounded queue, close idempotency, and tool dispatch tests without network access.

14.3 Property/state tests

Generate event permutations that preserve required per-item constraints and verify:

- no panic;
- terminal states remain terminal;
- duplicate final events are idempotent;
- output transcript never resurrects truncated text;
- one tool `call_id` executes at most once;
- canonical IDs remain stable.

14.4 Persistence tests

- no persistence on raw deltas;
- final user transcript persists once;
- interrupted assistant output stores only the heard/truncated portion;
- tool call and output persist atomically or with recoverable status;
- reconnect rehydrates only stable state;
- media references are immutable and deep-copy safe.

14.5 HTTP/bootstrap tests

- unauthenticated and CSRF failures;
- Origin/CORS checks;
- SDP body limits;
- upstream non-2xx mapping;
- missing `Location`/call ID;
- duplicate bootstrap idempotency;
- client-secret TTL validation;
- per-user quota;
- sideband-required readiness timeout;
- no secrets in response/log snapshots.

Use an HTTP recorder or fake upstream, not live OpenAI, for the majority of tests.

14.6 Browser tests

Use Playwright with fake media devices where possible:

- permission accepted/denied;
- user-gesture connection;
- microphone mute/unmute;
- remote track attachment;
- data-channel event parsing;
- close cleanup;
- sideband-ready gating;
- network disconnect UI;
- device change;
- captions under high delta rate.

A gated live integration suite should use a spend cap and explicit environment flag.

14.7 Audio proxy tests

For optional backend media proxy:

- deterministic resampling vectors;
- PCM16 saturation/endian tests;
- sequence loss/reordering behavior;
- playback ring-buffer underflow/overflow;
- exact sample-to-millisecond truncation;
- `WebSocket.bufferedAmount` thresholds;
- Base64 round trip;
- long-run memory/CPU soak;
- interruption under jitter.

14.8 Compatibility tests

The new packages should not change:

- existing `engine.Engine` behavior;
- existing Session append/next behavior;
- current tool-loop tests;
- current EventSink contracts;
- Chat/Responses provider selection.

15. Incremental rollout plan

Phase 0: establish a clean baseline

- restore or identify the missing `pkg/inference/toolloop/enginebuilder` package in the full development checkout;
- run existing unit/integration tests with the requested Go toolchain;
- record public API compatibility expectations;
- add feature flags for live functionality.

Phase 1: provider-neutral live core

Deliver:

- `pkg/inference/live` lifecycle interfaces;
- typed command queue and state machine;
- fake transport;
- canonical live event definitions;
- reducer/effect test harness;
- no browser or live OpenAI dependency.

Exit criteria:

- lifecycle and concurrency tests pass under race detection;
- bounded queues and close semantics are documented;
- existing code is unaffected.

Phase 2: server WebSocket adapter and protocol validation

Implement the OpenAI server WebSocket transport first as an engineering test harness, even though it is not the default browser topology. It gives Geppetto complete access to protocol events and simplifies fixture capture, tool integration, and reducer validation.

Deliver:

- exact event types;
- server WebSocket connection;

- audio append/output handling for test files or a CLI harness;
- tool dispatcher;
- interruption/truncate tests;
- usage/rate-limit metrics.

This phase validates protocol semantics before adding browser signaling.

Phase 3: unified WebRTC bootstrap and browser demo

Deliver:

- authenticated `/api/live/openai/webrtc` endpoint;
- browser `RTCPeerConnection` client;
- remote audio playback and microphone controls;
- sideband attachment before ready;
- final transcript captions;
- close/status endpoints;
- no recording;
- one configuration profile.

Phase 4: production sideband control

Deliver:

- privileged tool execution;
- per-tool authorization/confirmation;
- canonical event mapping;
- stable persistence barriers;
- sideband loss policy;
- observability dashboards;
- quotas and audit events.

Phase 5: chained voice

Deliver:

- true Realtime transcription adapter using `gpt-live-transcribe` or configured equivalent;
- integration with existing Geppetto Session/Engine/tool loop;
- streaming speech generation;
- barge-in/cancel policy;
- explicit transcript/policy checkpoints.

This replaces the current file-chunk “streaming” transcription path for live microphone use rather than modifying it in place.

Phase 6: hardening and optional media gateway

- 55-minute rollover;
- reconnect/summary rehydration;
- load/soak testing;
- browser network/device matrix;
- cost controls;
- recording/compliance design if required;
- backend media proxy only after a concrete requirement and capacity model.

Recommended first production scope

Keep the first release deliberately narrow:

```
transport:      direct browser WebRTC
bootstrap:      unified SDP
sideband:       required when tools are enabled
turn detection: server_vad
recording:      disabled
persistence:    final transcripts + tool barriers only
models/voice:   one versioned configuration profile
session length: capped below provider maximum
fallback:       text UI and explicit reconnect
```

16. Decision record and implementation checklist

16.1 Architecture decision record

ADR-1: Realtime is not an Engine

Status: recommended

Decision: introduce `pkg/inference/live` and `LiveSessionFactory`.

Reason: lifecycle and concurrency differ fundamentally from one finite inference.

Consequence: some shared utilities move below engine/live boundaries; callers choose finite or live explicitly.

ADR-2: WebRTC is the default browser media transport

Status: recommended

Decision: browser audio flows directly to OpenAI.

Reason: official recommendation, lower media implementation burden, better interruption/output buffering, lower backend load.

Consequence: raw audio is not available to Geppetto unless a separate recording path is added.

ADR-3: Unified SDP bootstrap is the default tool-enabled bootstrap

Status: recommended

Decision: browser posts offer SDP to Geppetto; Geppetto creates the call and attaches sideband before returning ready.

Reason: simpler correlation and no sideband attach race.

Consequence: backend is in the setup critical path, though not the media path.

ADR-4: Sideband owns privileged business logic

Status: recommended

Decision: tools, policy, and durable state stay on Geppetto.

Reason: client code and credentials are untrusted; OpenAI explicitly provides sideband for server control.

Consequence: sideband availability becomes a production dependency for tool-enabled sessions.

ADR-5: EventSink is observational, not a control bus

Status: recommended

Decision: reliable effects use internal queues; canonical events remain best-effort.

Reason: existing sink behavior ignores sink failures and warns against durable state from partial events.

Consequence: live code has explicit command/effect plumbing.

ADR-6: Persist semantic barriers, not audio deltas

Status: recommended

Decision: store final transcripts, completed/truncated items, and tool barriers.

Reason: deltas are high-volume, reorderable, interruption-sensitive, and potentially sensitive.

Consequence: exact audio replay requires a separate opt-in recording subsystem.

ADR-7: Support native and chained modes

Status: recommended

Decision: share product vocabulary but keep internal pipelines separate.

Reason: natural conversation and deterministic workflows are both valid requirements.

Consequence: capability profiles must state which mode, transport, and persistence semantics apply.

16.2 Implementation checklist

Domain and concurrency

- ☐ Define live states and terminal semantics.
- ☐ Implement one read pump and one write pump.
- ☐ Bound every queue and document overflow behavior.
- ☐ Make `Close` idempotent.
- ☐

Add race-enabled tests.

OpenAI adapter

- ☐ Define exact client/server event structs.
- ☐ Implement client-secret REST call.
- ☐ Implement unified multipart SDP call.
- ☐ Capture and protect `call_id`.
- ☐ Implement sideband WebSocket attachment.
- ☐ Add pure reducer and JSONL fixtures.
- ☐ Map errors through client `event_id` where available.

Tools

- ☐ Reuse registry/executor and schema definitions.
- ☐ Deduplicate by `call_id`.
- ☐ Reauthorize every tool invocation.
- ☐ Send `function_call_output` then `response.create`.
- ☐ Persist tool barriers.

Browser

- ☐ User-gesture-gated connection.
- ☐ Microphone/device controls.
- ☐ Remote track playback.
- ☐ Data-channel event parsing.
- ☐ Sideband-ready microphone gate.
- ☐ Clean close and track stop.
- ☐ Accessibility captions and connection status.

Security

- ☐ Permanent key only on backend.
- ☐ Client-secret TTL 30-60 seconds by default.

- ☐ Auth/CSRF/Origin/CORS checks.
- ☐ Concurrency/spend quotas.
- ☐ Redaction of secrets, SDP, audio, and transcripts.
- ☐ Explicit recording consent and retention if added.

Persistence and events

- ☐ Add live canonical event types.
- ☐ Keep raw audio off EventSink.
- ☐ Implement provider ConversationMirror.
- ☐ Persist final/transformed state after interruption.
- ☐ Mark transcript source and authority.

Operations

- ☐ Bootstrap/sideband/response/tool metrics.
- ☐ Browser WebRTC aggregate stats.
- ☐ Session rollover below 60 minutes.
- ☐ Reconnect/rehydration policy.
- ☐ Cost and active-session limits.
- ☐ Kill switch and feature flags.

17. Appendices and references

Appendix A: OpenAI event mapping starter table

Provider event	Canonical handling	Durable?
session.created	live-session-started after local readiness rules	session record
session.updated	live-session-updated	selected config snapshot
input_audio_buffer.speech_started	audio-input-started ; interrupt UI	no
input_audio_buffer.speech_stopped	audio-input-stopped	no
input_audio_buffer.committed	audio-input-committed	pending barrier
conversation.item.input_audio_transcription.delta	input-transcript-delta	no
conversation.item.input_audio_transcription.completed	input-transcript-finished	yes
response.created	response-started	no
response.output_audio.delta	media plane only for WebSocket	no
response.output_audio_transcript.delta	output-transcript-delta	no
response.output_audio_transcript.done	output-transcript-finished	candidate barrier
response.function_call_arguments.delta	internal assembler; optional debug event	no
response.function_call_arguments.done	reliable tool-dispatch effect	tool barrier
conversation.item.done	item-finished; update mirror	yes when stable
response.cancelled	response-cancelled/interrupted	persist truncation result
response.done	response-finished + usage	yes
rate_limits.updated	rate-limits-updated metric/event	aggregate only
error	canonical error; correlate by event_id	terminal or audit depending severity

The adapter must tolerate additional provider events and preserve unknown event type/ID in a bounded debug record without failing the entire session unless protocol correctness requires it.

Appendix B: Example configuration profile

```
api:
  type: openai-realtime

realtime:
  profile: support-default-v1
  model: ${OPENAI_REALTIME_MODEL}
  voice: marin
  transport: webrtc-direct
```

```
bootstrap_mode: unified_sdp
sideband_required: true
output_modalities: [audio]

turn_detection:
  type: server_vad
  create_response: true
  interrupt_response: true
  prefix_padding_ms: 300
  silence_duration_ms: 500

input_transcription:
  enabled: true
  model: gpt-live-transcribe
  language: en

noise_reduction:
  type: near_field

client_secret_ttl: 45s
max_session_age: 55m
rollover_lead: 2m
persist:
  final_input_transcripts: true
  final_output_transcripts: true
  tool_barriers: true
  raw_audio: false
```

Exact model/voice values must be validated against the active account and current API documentation.

Appendix C: Repository evidence map

Finding	Repository evidence
finite engine contract	<code>pkg/inference/engine/engine.go:9-15</code>
optional finite result wrapper	<code>pkg/inference/engine/run_with_result.go:26-103</code>
blocking session runner builder	<code>pkg/inference/session/builder.go:9-20</code>
one-inference execution handle	<code>pkg/inference/session/execution.go:13-84</code>
stable session ID and append-only snapshots	<code>pkg/inference/session/session.go:21-35</code>
clone latest Turn for next prompt	<code>pkg/inference/session/session.go:53-103</code>
asynchronous finite StartInference	<code>pkg/inference/session/session.go:189-281</code>
EventSink use and durability warning	<code>pkg/events/sink.go:3-18</code>
best-effort context fanout	<code>pkg/events/context.go:15-48</code>
canonical correlation IDs	<code>pkg/events/correlation.go:3-20</code>
current event vocabulary	<code>pkg/events/chat-events.go:13-96</code>
current Turn/block kinds omit audio	<code>pkg/turns/block_kind_gen.go:13-21</code>
shallow nested clone concern	<code>pkg/turns/types.go:28-62</code>
finite tool-loop orchestration	<code>pkg/inference/toolloop/loop.go:92-174</code>
reusable registry/executor	<code>pkg/inference/tools/registry.go:8-18</code> ; <code>executor.go:7-11</code>
factory supports finite APIs only	<code>pkg/inference/engine/factory/factory.go:105-197</code>
Chat Stream is response streaming	<code>pkg/steps/ai/settings/settings-chat.go:22-61</code>
file-chunk transcription implementation	<code>pkg/steps/ai/openai/transcribe.go:300-459</code>
session-centered JS design note	<code>ttp/2026/06/02/.../01-session-centered-javascript-api-design-and-implementation-guide.md</code>
JS streaming event design note	<code>ttp/2026/06/01/.../01-geppetto-js-streaming-events-design-and-implementation-guide.md</code>

Appendix D: Key failure modes and controls

Failure mode	Control
permanent key exposed in browser	server-only standard key; ephemeral/unified bootstrap
token reused to create extra calls	very short TTL, issuance quotas, application binding, monitoring
client overrides session config	tools authorize independently; sideband monitors; proxy for strict isolation
tool executes twice	<code>call_id</code> idempotency store
sideband attaches too late	unified bootstrap or microphone gate
output stored beyond interruption	post-truncation persistence barrier
WebSocket playback drifts	sample-count ring buffer and deterministic truncate tests
event subscriber blocks media	media plane separate; best-effort EventSink
unbounded memory	bounded command/event/audio queues
60-minute provider termination	graceful rollover before maximum
full transcript/audio leaks to logs	redaction, payload caps, data-classification defaults
reconnect replays partial state	restore stable summaries/items only
model instructions treated as permissions	backend authorization for every action

References

All OpenAI references were reviewed on 2026-08-11.

- **OAI-WEBRTC** - OpenAI, [Realtime API with WebRTC](#).
- **OAI-WEBSOCKET** - OpenAI, [Realtime API with WebSocket](#).
- **OAI-CONVERSATIONS** - OpenAI, [Realtime conversations](#).
- **OAI-SIDEBAND** - OpenAI, [Webhooks and server-side controls](#).
- **OAI-CLIENT-SECRETS** - OpenAI API Reference, [Create Realtime client secret](#).
- **OAI-VAD** - OpenAI, [Voice activity detection \(VAD\)](#).
- **OAI-TRANSCRIPTION** - OpenAI, [Realtime transcription](#).
- **OAI-VOICE-AGENTS** - OpenAI, [Voice agents](#).
- **OAI-COSTS** - OpenAI, [Managing costs](#).
- **OAI-REALTIME-MODELS** - OpenAI, [Using realtime models](#).
- **OAI-GO-SDK** - OpenAI, [Official Go SDK](#).